

Metanoia: when your agent becomes the buyer

In plain words: you give an AI a budget, it finds and buys the best subscription for you, and it can never spend a dollar you did **not allow**. You hand the agent a spending *mandate* instead of your card; it shops a marketplace of API and software subscriptions, and Juspay Hyperswitch settles the real payment in the sandbox. The agent only suggests. Plain, rule-based server code makes the final call and does the paying. Written by Aayush Shrestha.

1 The industry, and why I picked it

Vertical: developer and SaaS subscriptions, bought by an agent. I picked this because it is where payments are about to change shape. The interesting question in US commerce is no longer how a human checks out. It is **what happens when software does the buying**. Subscriptions are the ideal first surface. They are recurring, priced, comparable, and machine-describable, with a price, a throughput number, an uptime number, and feature flags. That is exactly what an agent can reason over and a deterministic ranker can score.

Why the US market shaped the choices. US SaaS is card-first and mandate-friendly. The Hyperswitch US connectors (Stripe, Adyen, Braintree) support `setup_future_usage` and customer-acceptance mandates for off-session recurring, with no local-rail complexity such as UPI or iDEAL to model. So the integration centers on **cards plus a saved-card mandate**, which is the real US subscription pattern.

*The thesis in one line:
the model proposes, the server decides, Hyperswitch settles.
An LLM can research and recommend. It can never set a price or move money.*

Why an agent and a mandate together? Because an agent good enough to buy for you is also good enough to be talked into overspending. So authority never lives in the model.

2 The flows this industry needs

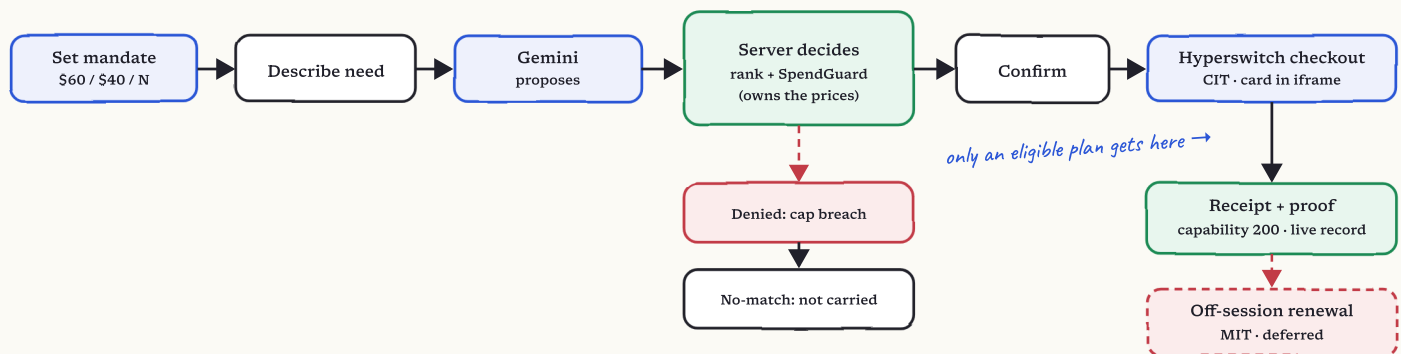


FIG 1. END-TO-END FLOW. GREEN IS DETERMINISTIC AUTHORITY; RED DASHED IS DEFERRED OR REFUSED, WHERE THE CARD IS NEVER TOUCHED.

A subscription business needs a chain of flows, and here is where I focused. First, **discovery and comparison**. Then **mandate-gated selection**. Then a **customer-initiated checkout (CIT)**. Then **proof that the thing works**. After that, **off-session recurring (MIT)**, plus **webhooks and reconciliation**, and finally **refunds and cancellation**. I built the spine from discovery through checkout end to end against the live sandbox. Recurring and MIT are designed and coded, but connector-blocked (see section 4).

3 What I built, what I deferred, and why

CAPABILITY	STATE	REASONING
Mandate, deterministic decide() , and SpendGuard	LIVE	A budget guarantee is only real if the server enforces it before any charge. An over-budget purchase is impossible through the app, and a refusal is a first-class, audited outcome.
Deterministic ranking over a 30-offer catalog (10 capabilities)	LIVE	Scoring is reproducible and auditable, computed on server-owned numbers rather than model opinion. Same inputs give the same ranking.
CIT checkout through Hyperswitch Unified Checkout (Fauxpay)	LIVE	A real sandbox payment settles. The card is entered in Hyperswitch's iframe and never touches my server.
Capability proof and a live payment record on the receipt	LIVE	After paying, an authenticated call returns 200, and I pull the real payment record from the Hyperswitch API, the same fields the dashboard shows. Nothing is faked.
Refunds, cancellation, resubscribe, and per-session isolation	LIVE	Refunds are server-side, idempotent, and owned-only, verified by retrieval. Each browser is its own tenant.
Deployed on Cloud Run, Cloud SQL, Vertex, and Secret Manager	LIVE	Hosted and sharable. Secrets live in Secret Manager, never in code.
Webhook receiver (HMAC verify, atomic settle)	LIVE	A valid signature returns 200; a bad or missing one returns 401. Verified on the deployed endpoint.
Off-session MIT and saved-card recurring	BLOCKED	Coded with <code>setup_future_usage</code> , <code>customer_acceptance</code> , and <code>recurring_details</code> . Routing the card to Stripe returns <code>UE_9000</code> , because the account needs raw-card API access, a Stripe-side grant. Fauxpay returns no reusable payment method. So MIT is proven in code, not yet in settlement.
Automatic renewal scheduler	DEFERRED	Renewal is manual and on demand today, and it re-runs SpendGuard. A background scheduler is the next step once MIT clears.
Webhook delivery and settlement	LIVE	Signed Hyperswitch <code>payment_succeeded</code> events reach an HMAC-verifying Vercel ingress, which forwards the untouched body to the Cloud Run receiver; Cloud Run verifies the HMAC again and settles the known payment in Cloud SQL (deduped by <code>event_id</code> , marked processed). Confirmed in the Vercel and Cloud Run logs and the events table. Direct <code>*.run.app</code> delivery does not land and the low-level transport reason is unconfirmed; unknown-payment events are retained, not dropped.
AP2 cryptographic mandate signatures, and x402	DEFERRED	The AP2 Intent and Cart shapes are modeled and enforced app-side. Signed JWT mandates are on the roadmap.

4 Integration and payment-method choices, with the reasoning

Unified Checkout, not raw card fields. The card is collected inside Hyperswitch's iframe, so the PAN and CVV never touch my server and I stay outside PCI scope. My server only ever holds the **publishable key** and a per-payment `client_secret`. The secret key stays server-side.

Connectors are processors, not payment methods. I never show Stripe, Adyen, or Fauxpay as a customer choice. The customer picks a *card*, and routing to a processor is a server decision. For US subscriptions the payment method that matters is a **card plus a saved-card mandate** for recurring.

Fauxpay for CIT, Stripe reserved for MIT. Fauxpay is a reliable dummy connector, so checkout always settles for a clean demo. Real off-session recurring needs a reusable `payment_method_id`, which only a real connector such as Stripe-test returns. That is why Stripe is wired for the mandate path.

Idempotency is a property of the payment id. I mint `pay_ = sha256(customer:plan:period)`, so a retry reuses the same intent instead of charging twice. A duplicate-id error self-heals: it reuses a live intent, or mints a fresh id for a dead one. Confirming twice records *one* subscription.

Webhooks: verify, then settle atomically. I check the raw-body HMAC (SHA-512/256, timing-safe), then run one DB transaction: dedupe by `event_id`, settle, upsert the subscription, and issue the credential. Unknown-payment events are retained, never dropped, so out-of-order delivery stays safe.

US subscription integration is Unified Checkout for cards, off-session mandates for recurring, stable-id idempotency, and signed webhooks as the source of truth.

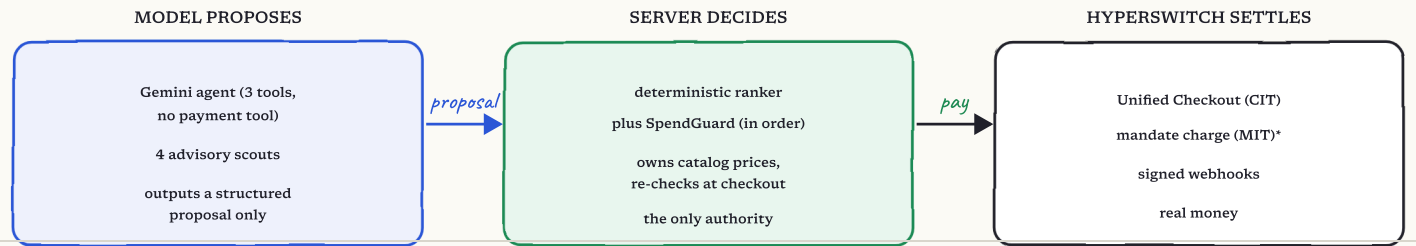


FIG 2. THE AUTHORITY BOUNDARY. A PROPOSAL IS THE ONLY THING THAT CROSSES FROM MODEL TO SERVER, AND MONEY ONLY CROSSES FROM SERVER TO HYPERSWITCH.

5 How the prototype fits together, end to end

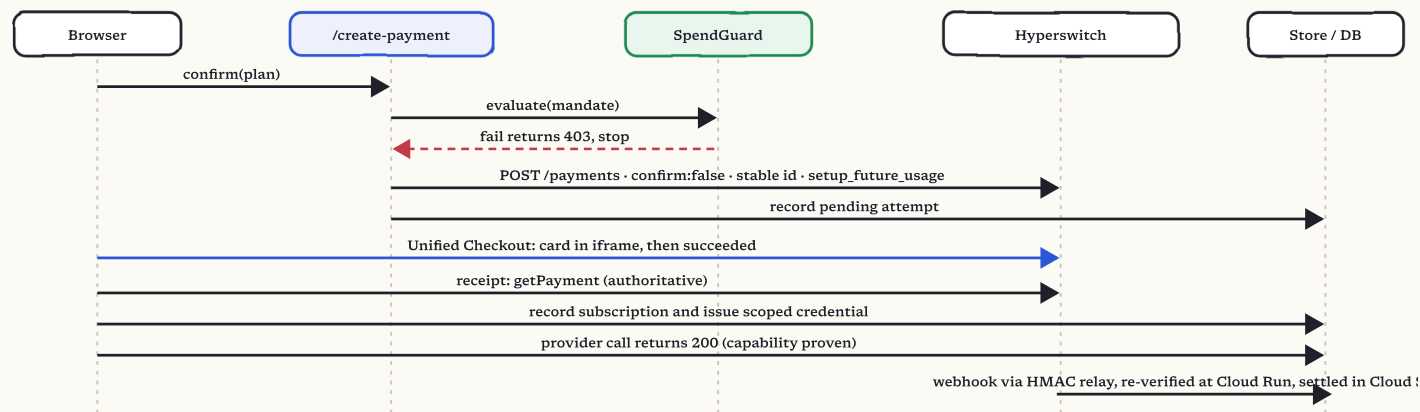


FIG 3. THE CUSTOMER-INITIATED PAYMENT SEQUENCE. SPENDGUARD RUNS AGAIN ON THE SERVER BEFORE HYPERSWITCH IS EVER CALLED.

The stack. Next.js 16 (App Router) runs on **Cloud Run**. Gemini 3.1 Pro and 2.5 Flash run on **Vertex AI** through the AI SDK tool loop. **Cloud SQL** Postgres is reached through the IAM connector, with no public IP. Secrets live in **Secret Manager**. Juspay **Hyperswitch** settles in the sandbox. A dual-backend Store (in-memory for dev and tests, Postgres in production) makes correctness a database property: an idempotent `payment_id` primary key, one credential per plan, and webhook `event_id` dedupe.

Safety and trust. Untrusted input, meaning the request and public GitHub metadata, is wrapped and labeled “data, not instructions.” The agent has no payment tool. The GitHub fetch is SSRF-guarded and only accepts `github.com/owner/repo`. Preference memory lives in a separate, consent-gated schema, never mixed with payments.

If a reviewer trusts one thing, trust this: the number that gets charged is computed by deterministic, unit-tested server code, on data the model never authored.

How I would finish it. First, get raw-card API access on the Stripe test account, or self-host Hyperswitch, to unblock the reusable `payment_method_id` and prove off-session MIT. Second, add the renewal scheduler on top of the already-tested, pending-first, idempotent renewal path, now that the webhook settlement loop is closed. Third, sign the AP2 Intent and Cart mandates.

91 tests green. Type-check and lint clean. A real sandbox payment completes end to end on the live URL.